

学号： 2103200423



沈阳建筑大学

智能手机应用程序开发作业

2023 年 秋 季学期

学 院： 计算机科学与工程学院

专业班级： 计算机 2101 班

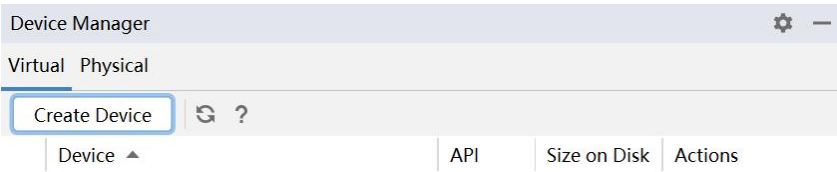
姓 名： 徐心雨

一、第一章

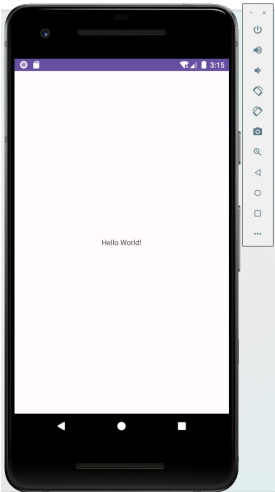
1-1 下载最新版本的 Android Studio，安装后如果需要更新 Gradle，则将其更新到最新版本。



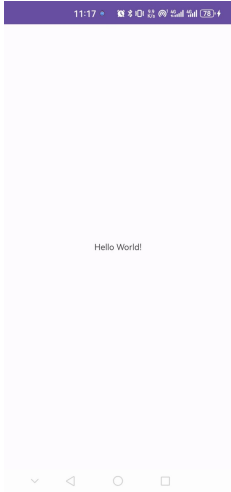
1-2 创建一个任意类型的手机 App 工程，分别使用虚拟设备和物理设备运行。



(1) 虚拟设备



(2) 物理设备



二、第二章

2-2 实现多选题的界面效果。

(1) 代码及注释

```
protected void onCreate(Bundle savedInstanceState) {  
    super.onCreate(savedInstanceState);
```

```

setContentView(R.layout.activity_main);
final TextView resultTextView = findViewById(R.id.text_view_result);
CheckBox checkBox1 = findViewById(R.id.check_box1);
CheckBox checkBox2 = findViewById(R.id.check_box2);
CheckBox checkBox3 = findViewById(R.id.check_box3);
CheckBox checkBox4 = findViewById(R.id.check_box4);
CheckBox checkBox5 = findViewById(R.id.check_box5);
CheckBox checkBox6 = findViewById(R.id.check_box6);
CheckBox checkBox7 = findViewById(R.id.check_box7);
CheckBox checkBox8 = findViewById(R.id.check_box8);
CheckBox checkBox9 = findViewById(R.id.check_box9);
Button submitButton = findViewById(R.id.button_submit);
//CheckBox 设置监听器
checkBox1.setOnCheckedChangeListener(this);
checkBox2.setOnCheckedChangeListener(this);
checkBox3.setOnCheckedChangeListener(this);
checkBox4.setOnCheckedChangeListener(this);
checkBox5.setOnCheckedChangeListener(this);
checkBox6.setOnCheckedChangeListener(this);
checkBox7.setOnCheckedChangeListener(this);
checkBox8.setOnCheckedChangeListener(this);
checkBox9.setOnCheckedChangeListener(this);
//Button 设置监听器
submitButton.setOnClickListener(v -> {
    StringBuilder sb = new StringBuilder();
    for (int i = 0; i < mHobbies.size(); i++) {
        //选择的添加到 StringBuilder 尾部
        if (i == (mHobbies.size() - 1)) {
            sb.append(mHobbies.get(i));
        } else {
            sb.append(mHobbies.get(i)).append("、");
        }
    }
    //显示选择结果
    if (sb.length() == 0) {
        resultTextView.setText("您还没有进行选择！");
    } else {
        resultTextView.setText("您选择了： " + sb + "。");
    }
}

```

```
});  
}
```

(2) 运行结果



2-3 实现选择出生日期的界面效果。

(1) 代码及注释

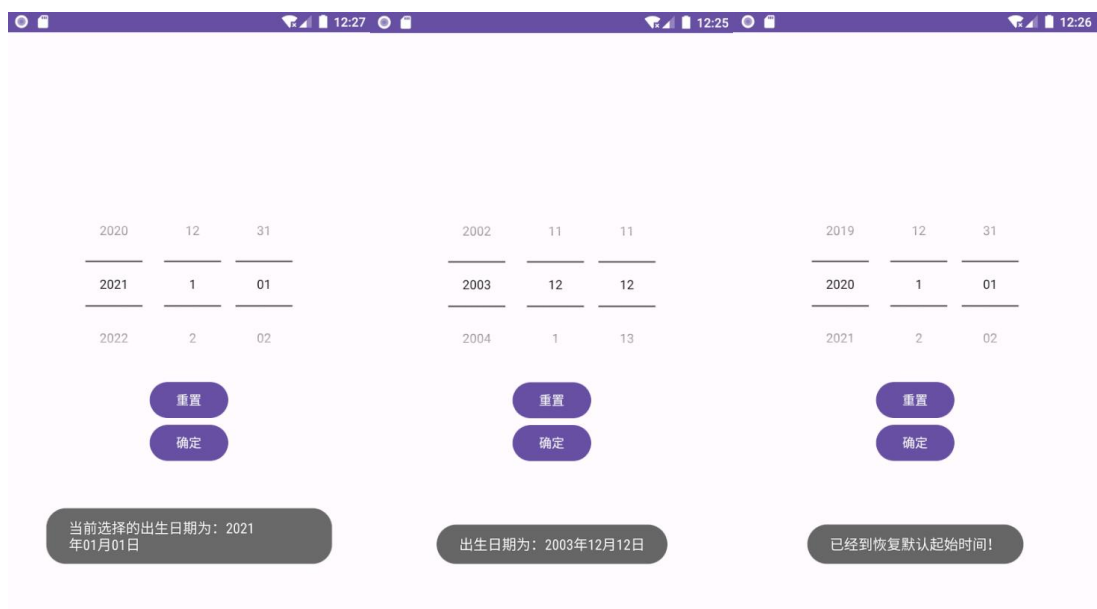
```
protected void onCreate(Bundle savedInstanceState) {  
    super.onCreate(savedInstanceState);  
    setContentView(R.layout.activity_main);  
    final DatePicker datePicker = findViewById(R.id.date_picker);  
    datePicker.updateDate(2021, 0, 1);  
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {  
        datePicker.setOnDateChangeListener((view, year, monthOfYear,  
dayOfMonth) -> {  
            Calendar calendar = Calendar.getInstance();  
            calendar.set(year, monthOfYear, dayOfMonth);  
            @SuppressWarnings("SimpleDateFormat") SimpleDateFormat  
simpleDateFormat = new SimpleDateFormat("yyyy 年 MM 月 dd 日");  
            Toast.makeText(getApplicationContext(), "当前选择的出生日期为：  
" + simpleDateFormat.format(calendar.getTime()),  
Toast.LENGTH_SHORT).show();  
        });  
    }  
    Button button1 = findViewById(R.id.button1);
```

```

        button1.setOnClickListener(v -> {
            datePicker.updateDate(2020, 0, 1);
            Toast.makeText(getApplicationContext(), "已经到恢复默认起始时间!",
Toast.LENGTH_LONG).show();
        });
        Button button2 = findViewById(R.id.button2);
        button2.setOnClickListener(v -> {
            Calendar calendar = Calendar.getInstance();
            calendar.set(datePicker.getYear(),                    datePicker.getMonth(),
datePicker.getDayOfMonth());
            @SuppressWarnings("SimpleDateFormat")                SimpleDateFormat
            simpleDateFormat = new SimpleDateFormat("yyyy 年 MM 月 dd 日");
            Toast.makeText(getApplicationContext(), " 出生日期为 : " +
            simpleDateFormat.format(calendar.getTime()), Toast.LENGTH_SHORT).show();
        });
    }

```

(2) 运行结果



三、第三章

3-1 实现注册界面的布局效果，至少包括用户名、密码、忘记密码、登录等控件。

(1) 代码及注释

```

<!--第 1 行-->
<TableRow>

```

```

<!--第 1 列-->
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="用户名" />
<!--第 2 列-->
<EditText
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:ems="10"
    android:inputType="textPersonName" />
</TableRow>
<!--第 2 行-->
<TableRow>
    <!--第 1 列-->
    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="密码" />
    <!--第 2 列-->
    <EditText
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:ems="10"
        android:inputType="textPassword" />
    <!--第 3 列-->
    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="注册账号" />
</TableRow>
<!--第 3 行-->
<TableRow>
    <!--第 2 列-->
    <TextView
        android:id="@+id/textView4"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_column="1"

```

```

        android:gravity="end"
        android:text="忘记密码" />
</TableRow>
<!-- 第 4 行 -->
<TableRow>
    <!-- 第 1 列和第 2 列合并 -->
    <Button
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_span="2"
        android:text="登录" />
</TableRow>
</TableLayout>

```

(2) 运行结果



四、第四章

4-3 实现京东首页的轮播广告效果，至少包括 3 个产品广告的图片。

(1) 代码及注释

```

//页面滚动事件
@SuppressLint("SetTextI18n")
public void onPageScrolled(int position, float positionOffset, int positionOffsetPixels) {
    //mPagePositionTextView.setText(" 页面索引号:" + position + " 偏移百分比:" + positionOffset + " 偏移像素:" + positionOffsetPixels);
}

```

```

}
//页面滚动状态改变事件
public void onPageScrollStateChanged(int state) {
    switch (state) {
        case ViewPager.SCROLL_STATE_IDLE:
            mPageStateTextView.setText("空闲状态");
            break;
        case ViewPager.SCROLL_STATE_DRAGGING:
            mPageStateTextView.setText("拖动状态");
            break;
        case ViewPager.SCROLL_STATE_SETTLING:
            mPageStateTextView.setText("结束状态");
            break;
    }
}
}
public class WelcomePagerAdapter extends PagerAdapter {
    private ArrayList<View> mPagerViews;//分页视图的数组列表
    //构造方法
    WelcomePagerAdapter(ArrayList<View> pagerViews) {
        mPagerViews = pagerViews;
    }
    //获取分页视图的数量
    public int getCount() {
        return mPagerViews.size();
    }
    //判断是否由对象生成界面。
    public boolean isViewFromObject(View view, Object object) {
        return view == object;
    }
    //显示分页视图或缓存分页时进行布局的初始化。
    public Object instantiateItem(ViewGroup container, int position) {
        container.addView(mPagerViews.get(position));
        return mPagerViews.get(position);
    }
    //销毁分页视图时移除相应的分页。
    public void destroyItem(ViewGroup container, int position, Object object) {
        container.removeView(mPagerViews.get(position));
    }
}
}

```

(2) 运行结果



五、第五章

5-1 实现登录后显示用户名的效果。输入用户名、密码后登录，启动一个新的 Activity 显示用户名。

(1) 代码及注释

```
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);
    usernameEditText = findViewById(R.id.usernameEditText);
    passwordEditText = findViewById(R.id.passwordEditText);
    Button loginButton = findViewById(R.id.loginButton);
    loginButton.setOnClickListener(v -> {
        String username = usernameEditText.getText().toString();
        String password = passwordEditText.getText().toString();
        // 进行用户名和密码验证的逻辑
        if (isValidCredentials(username, password)) {
            // 登录成功，启动新的活动并传递用户名
            Intent intent = new Intent(MainActivity.this, LoggedInActivity.class);
            intent.putExtra("username", username);
            startActivity(intent);
        } // 登录失败，可以显示错误消息或采取其他操作
    });
}

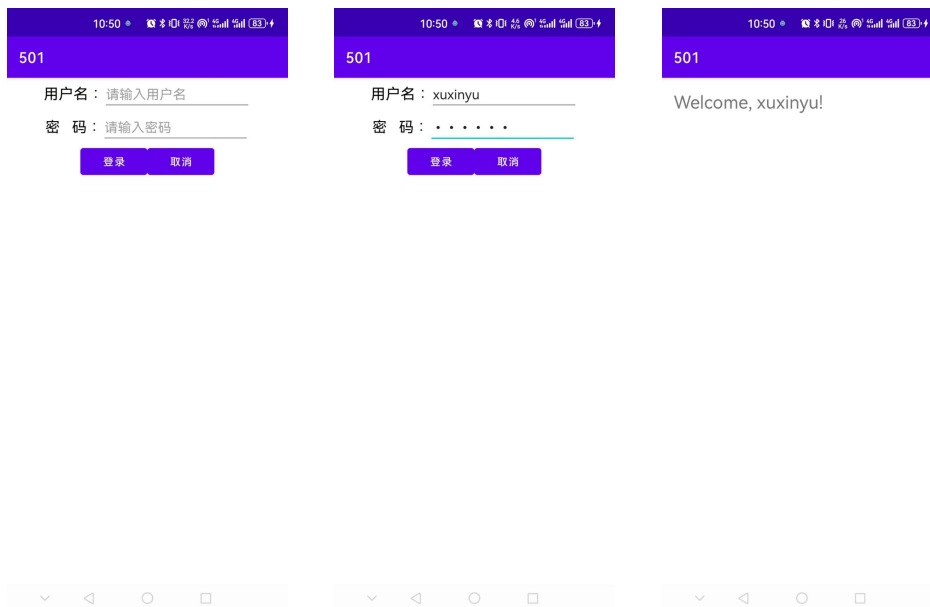
private boolean isValidCredentials(String username, String password) {
    return username.equals("xuxinyu") && password.equals("123456");
}
```

```

public class LoggedInActivity extends AppCompatActivity {
    @SuppressWarnings("SetTextI18n")
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_logged_in);
        TextView usernameTextView = findViewById(R.id.username);
        // 获取传递过来的用户名
        String username = getIntent().getStringExtra("username");
        // 设置用户名到 TextView 中显示
        usernameTextView.setText("Welcome, " + username + "!");
    }
}

```

(2) 运行结果



六、第六章

6-3 通过广播接收器实现两个 App 之间通过自定义广播进行数据通信。

(1) 代码及注释

SenderApp

```

public class MainActivity extends AppCompatActivity {
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        //发送标准广播
    }
}

```

```

        findViewById(R.id.button).setOnClickListener(new
View.OnClickListener() {
            public void onClick(View v) {
                Intent intent=new Intent();
                intent.setAction("MyBroadcastReceiver.Custom");
                intent.putExtra("info","这是徐心雨发送的广播!!!");

intent.addFlags(Intent.FLAG_ACTIVITY_PREVIOUS_IS_TOP);
                sendBroadcast(intent);
            }
        });
    }
}

public class MyReceiver1 extends BroadcastReceiver {
    private static final String TAG = "MyBroadcastReceiver";
    public void onReceive(Context context, Intent intent) {
        Log.d(TAG, "-----Receivers1-----");
        Log.d(TAG, "Action: " + intent.getAction());
        Log.d(TAG, "URI: " + intent.toUri(Intent.URI_INTENT_SCHEME));
        Log.d(TAG, intent.getStringExtra("info"));
    }
}

```

ResiverApp

```

public class MainActivity extends AppCompatActivity {
    private MyReceiver2 myReceiver = new MyReceiver2();
    private boolean mRegistered = false;
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);//注册广播接收器
        IntentFilter intentFilter = new IntentFilter();

intentFilter.addAction(ConnectivityManager.CONNECTIVITY_ACTION);
        intentFilter.addAction(Intent.ACTION_SCREEN_ON);
        intentFilter.addAction(Intent.ACTION_SCREEN_OFF);
        intentFilter.addAction(Intent.ACTION_USER_PRESENT);
        intentFilter.addAction(Intent.ACTION_BATTERY_OKAY);
        intentFilter.addAction(Intent.ACTION_BATTERY_LOW);
    }
}

```

```

        intentFilter.addAction(Intent.ACTION_BATTERY_CHANGED);
        registerReceiver(myReceiver, intentFilter);
        mRegistered = true;
    }
    protected void onDestroy() {
        super.onDestroy();
        if(mRegistered) {
            unregisterReceiver(myReceiver);
        }
    }
}

public class MyReceiver2 extends BroadcastReceiver {
    private static final String TAG = "MyBroadcastReceiver";
    public void onReceive(Context context, Intent intent) {
        // TODO: This method is called when the BroadcastReceiver is receiving
        // an Intent broadcast.
        Log.d(TAG, "-----Receiver-----");
        Log.d(TAG, "Action: " + intent.getAction());
        Log.d(TAG, "URI: " + intent.toUri(Intent.URI_INTENT_SCHEME));
        // 自定义广播
        if ("MyBroadcastReceiver.Custom".equals(intent.getAction())) {
            Log.d(TAG, intent.getStringExtra("info"));
        }
    }
}

```

(2) 运行结果



发送广播

发送广播

接收广播

```

D/MyBroadcastReceiver: -----Receiver-----
D/MyBroadcastReceiver: Action: MyBroadcastReceiver.Custom
D/MyBroadcastReceiver: URI: intent:#Intent;action=MyBroadcastReceiver
.Custom;launchFlags=0x1000010;component=com.example.resiverapp/
.MyReceiver2;S.info=%E8%BF%99%E6%98%AF%E5%BE%90%E5%BF%83%E9%9B%A8%E5%8F
%91%E9%80%81%E7%9A%84%E5%B9%BF%E6%92%AD%EF%BC%81%EF%BC%81%EF%BC%81;end
D/MyBroadcastReceiver: 这是徐心雨发送的广播!!!

```

七、第七章

7-2 使用 SQLite 实现记录登录时间的功能，并且能够查询登录的历史记录。

(1) 代码及注释

```
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);
    //控件定位
    helper=new DataBaseHelper(this); //传递上下文
    username=(EditText) findViewById(R.id.username);
    password=(EditText) findViewById(R.id.password);
    resign=(Button) findViewById(R.id.resign);
    login=(Button) findViewById(R.id.login);
    btn_new=(Button)findViewById(R.id.btn_new);
    //①以读写的方式打开数据库，一旦数据库的磁盘空间满了，数据库就只能读而不能写，倘若使用的是 getWritableDatabase() 方法就会出错
    //②还有一个 getReadableDatabase()方法也是以读写方式打开数据库，如果数据库的磁盘空间满了，就会打开失败，当打开失败后会继续尝试
    //以只读方式打开数据库。如果该问题成功解决，则只读数据库对象就会关闭，然后返回一个可读写的数据库对象。
    sqLiteDatabase = helper.getWritableDatabase();
    //设置按钮监听
    resign.setOnClickListener(this);
    login.setOnClickListener(this);
    btn_new.setOnClickListener(this);
}
public void onClick(View v) {
    int a = 0;
    if (v.getId() == R.id.resign)
        a = 1;
    else if (v.getId() == R.id.login)
        a = 2;
    else if (v.getId() == R.id.btn_new)
        a = 3;
    // 通过 id 来分配按钮事件
    if (a == 1) {
        //注册相关的弹窗设定，如果确定提交，则完成注册，数据存入数据库
        //AlertDialog: 对话框控件
        new AlertDialog.Builder(MainActivity.this).setTitle("系统提示")
            .setMessage("是否确定提交? ")
```

```

        //为确定按钮配置监听
        .setPositiveButton("    确    定    ", new
DialogInterface.OnClickListener() {
            public void onClick(DialogInterface arg0, int arg1) {
                // 获取 EditText 中的内容, 并将其转化成字符串型
后存入变量中

                name_str = username.getText().toString();
                paswd_str = password.getText().toString();
                time_str = DateUtil.getNowTimeDetail();
                //给数据表设置游标, Cursor 是一个游标, 以 name
字段为依据, query 是一种根据条件获取数据的方法
                Cursor                cursor                =
sqliteDatabase.query(Constants.TABLE_NAME, new String[]{"name"}, "name=?",
new String[]{name_str}, null, null, null);
                //如果游标找到了所需要的 name, 则返回已注册,
否则就利用之前写的 insert 方法插入数据
                if (cursor.getCount() != 0) {
                    Toast.makeText(MainActivity.this, "该用户已注
册!", Toast.LENGTH_SHORT).show();
                } else {
                    helper.insert(sqliteDatabase,                name_str,
paswd_str, time_str);
                    Toast.makeText(MainActivity.this, "注册成功,请
登录!", Toast.LENGTH_SHORT).show();
                }
            }
        }).setNegativeButton("    返    回    ", new
DialogInterface.OnClickListener() {

            public void onClick(DialogInterface arg0, int arg1) {
                //这个是点击返回后的操作, 因为不需要, 所以不
管他直接跳出就好。

            }
        }).show();
    }
    else if (a == 2) {
        //登录按钮功能, 上面解释过的已省略
        String user_str = username.getText().toString();

```

```

String psw_str = password.getText().toString();
//账号或密码为空时
if (user_str.equals("")) {
    Toast.makeText(this, " 账 号 或 密 码 不 能 为 空 ",
Toast.LENGTH_SHORT).show();
} else {
    Cursor cursor = sqLiteDatabase.query(Constants.TABLE_NAME,
new String[]{"password"}, "name=?", new String[]{user_str}, null, null, null);
    //游标的遍历, 寻找 name 对应的 password 的值
    if (cursor.moveToNext()) {
        @SuppressWarnings("Range") String psw_query =
cursor.getString(cursor.getColumnIndex("password"));
        //用户名对应的密码与输入的密码相同时
        if (psw_str.equals(psw_query)) {
            helper.insert(sqLiteDatabase, name_str, paswd_str,
time_str);
            Toast.makeText(this, " 登 录 成 功 !",
Toast.LENGTH_SHORT).show();
            //跳转到 successActivity 页面
            Intent intent = new Intent(MainActivity.this,
successActivity.class);
            //intent 会携带上 mstr 的值并以 username 命名
            intent.putExtra("username", mstr);
            //开始跳转事件
            startActivity(intent);
        }
        //密码输入错误时
        else {
            //跳转到 FaultActivity 页面
            Intent intent2 = new Intent(MainActivity.this,
FaultActivity.class);
            startActivity(intent2);
        }
    }
    //遍历完后发现在表中找不到游标携带的 name 的值时
    else {
        Toast.makeText(this, " 账 号 不 存 在 , 请 先 注 册 ! ",
Toast.LENGTH_SHORT).show();
    }
}

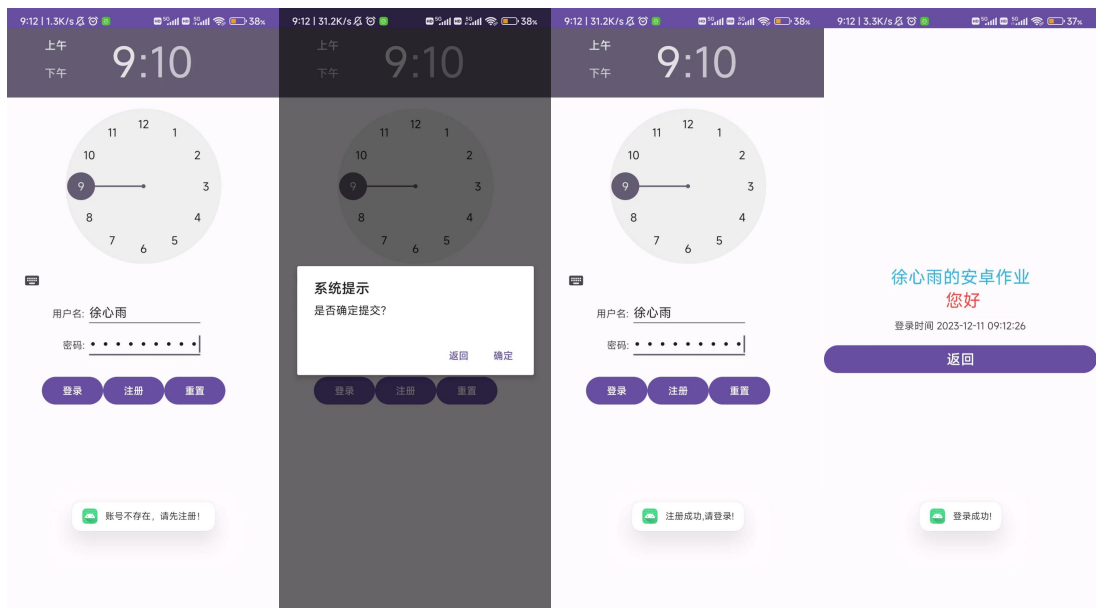
```

```

    }
} else if (a == 3){
    //重置按钮功能
    //将 EditText 的文本清空
    username.setText("");
    password.setText("");
}
}

```

(2) 运行结果



八、第八章

8-1 使用 Camera2 实现定时拍照的功能。

(1) 代码及注释

```

public class Util {
    // 创建文件
    public static File creatFile(String parentPath, String childPath, String
extension) {
        // 使用时间生成文件名
        SimpleDateFormat format = new
SimpleDateFormat("yyyyMMdd_hhmmss");
        String fileName = format.format(new Date());
        // 存储路径
        File storageDir = new File(parentPath, childPath);
        if (!storageDir.exists()) {

```

```

        storageDir.mkdirs();
    }
    // 创建文件
    File saveRecorderFile = new File(storageDir, "IMG_" + fileName + "." +
extension);
    if (saveRecorderFile.exists()) {
        saveRecorderFile.delete();
    }
    return saveRecorderFile;
}
// 将图片或视频显示在系统相册
public static void showInAlbum(Context context, String path) {
    // Android 中分割字符串需要在分隔符左右两侧加上中括号
    String[] str = path.split("[.]");
    // 获取扩展名对应的文件类型值
    String                mimeType                =
MimeTypeMap.getSingleton().getMimeTypeFromExtension(str[str.length - 1]);
    // 根据路径和扩展名的类型扫描媒体文件
    MediaScannerConnection.scanFile(context, new String[]{path}, new
String[]{mimeType}, null);
    // 将图片显示在系统相册内
    context.sendBroadcast(new
Intent(Intent.ACTION_MEDIA_SCANNER_SCAN_FILE, Uri.parse("file://" +
path)));
}
// 保存图片
public static byte[] saveImage(byte[] bytes, File file, int degree) {
    if (degree != 0) {
        // byte[]转 Bitmap
        Bitmap bitmap = BitmapFactory.decodeByteArray(bytes, 0,
bytes.length);
        // 旋转
        bitmap = rotateBitmapByDegree(bitmap, degree);
        // Bitmap 转 byte[]
        ByteArrayOutputStream baos = new ByteArrayOutputStream();
        bitmap.compress(Bitmap.CompressFormat.JPEG, 100, baos);
        bytes = baos.toByteArray();
    }
    // 写入文件

```

```

        FileOutputStream output = null;
        try {
            output = new FileOutputStream(file);
            output.write(bytes);
        } catch (IOException e) {
            e.printStackTrace();
        } finally {
            if (null != output) {
                try {
                    output.close();
                } catch (IOException e) {
                    e.printStackTrace();
                }
            }
        }
        return bytes;
    }
}
// 旋转位图
public static Bitmap rotateBitmapByDegree(Bitmap bitmap, int degree) {
    Bitmap returnBitmap = null;
    // 根据旋转角度，生成旋转矩阵
    Matrix matrix = new Matrix();
    matrix.postRotate(degree);
    try {
        // 将原始图片按照旋转矩阵进行旋转，并得到新的图片
        returnBitmap = Bitmap.createBitmap(bitmap, 0, 0,
        bitmap.getWidth(), bitmap.getHeight(), matrix, true);
    } catch (OutOfMemoryError e) {
    }
    if (returnBitmap == null) {
        returnBitmap = bitmap;
    }
    // 回收内存
    if (bitmap != returnBitmap && !bitmap.isRecycled()) {
        bitmap.recycle();
    }
    return returnBitmap;
}
// bitmap 转 byte[]

```

```

public static byte[] imageToBytes(Image image){
    ByteBuffer buffer = image.getPlanes()[0].getBuffer();
    byte[] bytes = new byte[buffer.remaining()];
    buffer.get(bytes);
    return bytes;
}
// 获取最佳尺寸
public static Size chooseOptimalSize(Context context, Size[] choices, int
previewWidth, int previewHeight, int maxPreviewWidth, int maxPreviewHeight,
Size aspectRatio) {
    Point displaySize = new Point();
    // 获取屏幕尺寸
    ((Activity)
context).getWindowManager().getDefaultDisplay().getSize(displaySize);
    int textureViewWidth = previewWidth;
    int textureViewHeight = previewHeight;
    int maxWidth = displaySize.x;
    int maxHeight = displaySize.y;
    if (maxWidth > maxPreviewWidth) { maxWidth = maxPreviewWidth; }
    if (maxHeight > maxPreviewHeight) { maxHeight = maxPreviewHeight; }
    List<Size> bigEnough = new ArrayList<>();//用于保存支持的不小于预览
    图像的 尺寸
    List<Size> notBigEnough = new ArrayList<>();//用于保存支持的小于预览
    图像的 尺寸
    int w = aspectRatio.getWidth();
    int h = aspectRatio.getHeight();
    // 将设备支持的尺寸分类保存
    for (Size option : choices) {
        if (option.getWidth() <= maxWidth && option.getHeight() <=
maxHeight && option.getHeight() == option.getWidth() * h / w) {
            if (option.getWidth() >= textureViewWidth &&
                option.getHeight() >= textureViewHeight) {
                bigEnough.add(option);
            } else {
                notBigEnough.add(option);
            }
        }
    }
}
// 选择足够大的最小的尺寸。如果没有足够大的尺寸，就从不够大的

```

尺寸中挑选最大的尺寸。

```
        if (bigEnough.size() > 0) {
            return Collections.min(bigEnough, new
Util.CompareSizesByArea());
        } else if (notBigEnough.size() > 0) {
            return Collections.max(notBigEnough, new
Util.CompareSizesByArea());
        } else {
            return choices[0];
        }
    }
    // 比较两个 Size 的大小的规则
    static class CompareSizesByArea implements Comparator<Size> {
        public int compare(Size lhs, Size rhs) {
            return Long.signum((long) lhs.getWidth() * lhs.getHeight() - (long)
rhs.getWidth() * rhs.getHeight());
        }
    }
}
```

(2) 运行结果

